```
    public String create(Request request, Token token) {
        var claimsBuilder = new JWTClaimsSet.Builder()    #A
                .subject(token.username)    #A
                .audience("https://localhost:4567")    #A
                .expirationTime(Date.from(token.expiry));
#A
        token.attributes.forEach(claimsBuilder::claim);    #A

        var header = new JWEHeader(JWEAlgorithm.DIR,     #B
                EncryptionMethod.A128CBC_HS256);    #B
        var jwt = new EncryptedJWT(header,
claimsBuilder.build());    #B

        try {
            var encrypter = new DirectEncrypter(encKey);
#C
            jwt.encrypt(encrypter);    #C
        } catch (JOSEException e) {
            throw new RuntimeException(e);
        }

        return jwt.serialize();    #D
    }

    @Override
    public void revoke(Request request, String tokenId) {
    }
}
```

#A Build the JWT claims set.
#B Create the JWE header and assemble the header and claims.
#C Encrypt the JWT using the AES key in direct encryption mode.
#D Return the Compact Serialization of the encrypted JWT.

Processing an encrypted JWT using the library is just as simple as creating one. First you parse the encrypted JWT